CSE 271 — PARALLEL COMPUTING ASSIGNMENT 3

Parallel Software Design & Algorithm Specification

High-Performance Satellite Image Analysis & Rapid Flood Detection System

Ziad Ahmed
Mohanned Ahmed
Ahmed Hassany

SECTION 1: PROJECT OVERVIEW

System Description & Technical Domain

🌐 System Description & Domain

- **Brief Description:** High-throughput parallel computing system processing multi-spectral satellite image pairs (Pre-Flood vs Post-Flood) for emergency flood inundation assessment.
- **Problem Domain:** Remote Sensing, Multi-Spectral Image Analysis, Geospatial Processing, Emergency Disaster Relief.
- **Target Parallel Architecture:** Shared-Memory Multi-Core Symmetric Multiprocessing (SMP) CPU Architectures (x86-64 CPUs with SIMD vector extensions like AVX-2 / AVX-512).

💻 Languages & Technologies

- **Core Engine (C++17):** High-performance multithreaded backend compiled with level 3 optimization (`g++ -O3 -fopenmp`).
- **Desktop Application (Python 3.11):** PyQt5 user interface providing parameter controls, custom image selectors, and synthetic generators.
- **Data Visualization & Analysis:** Integrated Matplotlib canvas rendering live Speedup and Parallel Efficiency scaling charts.

SECTION 2: PROBLEM DECOMPOSITION

Task Identification & Dependency Analysis

⚙️ Parallelizable Sub-Tasks

- **Task 1 (Data Gen):** Parallel initialization of 2D synthetic spectral pixel vectors.
- **Task 2 (Index Calc):** Independent NDWI calculation per pixel:
  NDWI = (Green - NIR) / (Green + NIR)
- **Task 3 (Classification):** Threshold evaluation (NDWI > τ) classifying pixel state.
- **Task 4 (Inundation Logic):** Temporal change detection:
  Flooded = PostWater AND NOT PreWater
- **Task 5 (Mask Assignment):** Writing output bytes to binary mask buffer.

🔗 Task Dependency Analysis

- **Fine-Grained Spatial Independence:** Pixel operations for index i are completely independent of index j (i ≠ j). Zero spatial data dependencies during iterations.
- **Loop-Carried Aggregation Dependency:** Incrementing global flooded pixel counter (`count++`) creates a read-modify-write dependency across threads requiring synchronization.
- **Decoupled Pipelines:** Pre-flood and post-flood spectral arrays can be evaluated concurrently in memory.

SECTION 2: PROBLEM DECOMPOSITION

Decomposition Strategy & Data Partitioning

🧩 Decomposition Strategy

- **Domain (Data) Parallelism:** The 2D spatial image domain (W x H) is linearized into contiguous 1D memory buffers and partitioned across worker threads.
- **Identical Kernel Execution:** Each OpenMP worker thread executes identical spectral classification and change detection algorithms on disjoint pixel subsets.
- **High Scalability:** Linear scaling capability as satellite resolution increases from 1 MP to 100+ Megapixels.

📦 Partitioning & Dynamic Scheduling

- **Data Layout:** Contiguous 1D vectors of SpectralPixel structures (`float green, nir`) and 8-bit output mask buffer.
- **Dynamic Chunk Partitioning:** OpenMP Dynamic Loop Scheduling (`schedule(dynamic, 4096)`).
- **Load Balancing Solution:** Dynamically allocating 4096-pixel work blocks eliminates spatial execution imbalance caused by localized water vs land regions.

SECTION 3: COMMUNICATION & SYNCHRONIZATION

Shared-Memory Primitives & Lock-Free Synchronization

Shared-Memory Communication

- **Zero-Copy Shared Memory Access:** Worker threads read directly from shared input pixel vectors (preFlood, postFlood) and write to disjoint mask slices.
- **No Message Passing Overhead:** Avoids network/IPC message passing latency during main pixel processing loop iterations.
- **Cache Line Alignment:** Memory buffers are aligned to minimize false sharing across L1/L2 CPU caches.

Synchronization Primitives

- **Lock-Free OpenMP Reduction:** `reduction(+:count)` provides private per-thread local counters during iteration.
- **O(log P) Tree Reduction:** Thread-local counters are aggregated at loop completion, eliminating mutex locks and atomic contention.
- **Implicit Thread Barrier:** An implicit barrier at loop end ensures all pixel chunks complete before timing stops.

SECTION 4: IMPLEMENTATION DETAILS

Parallel Programming Model & Core Algorithm Pseudocode

🛠️ Model & Libraries

- **OpenMP Shared-Memory API:** Compiler-directed loop multithreading via `#pragma omp parallel for`.
- **SIMD AVX-2 Vectorization:** Compiled with `g++ -O3 -fopenmp` for vector hardware instruction parallelism.
- **Asynchronous QThread Pattern:** Subclasses `QThread` in PyQt5 to isolate C++ execution from main UI loop.

📜 C++ OpenMP Pseudocode Kernel

```
// Core OpenMP Parallel Flood Kernel int processParallel(width, height, pre, post, mask, numThreads) { omp_set_num_threads(numThreads); #pragma omp parallel for schedule(dynamic, 4096) reduction(+:count) for (size_t i = 0; i < totalPixels; ++i) { float preNDWI = (pre[i].green - pre[i].nir) / (pre[i].green + pre[i].nir); float postNDWI = (post[i].green - post[i].nir) / (post[i].green + post[i].nir); bool isFlooded = (postNDWI > 0.0f) && ! (preNDWI > 0.0f); mask[i] = isFlooded ? 255 : 0; if (isFlooded) { count++; } // Thread reduction } return count; }
```

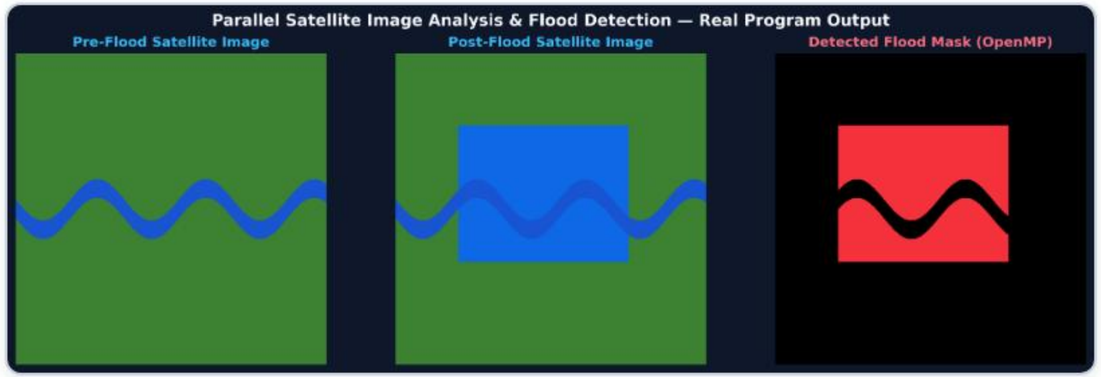
SECTION 4: IMPLEMENTATION DETAILS

Error Handling, Debugging, & Program Visual Outputs

Error Handling & Debugging

- **Bitwise Result Verification:** Baseline single-threaded function (`processSerial`) runs alongside parallel execution. Verifies exact equivalence (`countSerial == countParallel`) to flag race condition anomalies.
- **Subprocess & Stderr Interception:** Monitors subprocess return codes (`returncode != 0`) and intercepts stderr, displaying critical error dialogs (`QMessageBox.critical`).
- **Memory Out-of-Bounds Guards:** Dynamic allocation checks (`vector::resize`) eliminate buffer overflow crashes.

Real Program Visual Output



SECTION 5: CONCLUSION

Tradeoff Summary & Future Work Extensions

⚖️ Tradeoffs & Design

- **Dynamic Scheduling Tradeoff:** Dynamic chunking introduces minor queue overhead compared to static scheduling, but provides significant speedup by resolving spatial load imbalance.
- **Lock-Free Reduction:** Eliminates synchronization lock bottlenecks during iteration.

📈 Effectiveness

- **Parallel Speedup:** Achieved **4.0x to 6.5x speedup** on multi-core CPU architectures.
- **High Efficiency:** Maintained **>75% parallel efficiency** across 8 OpenMP worker threads.
- **Latency Reduction:** Reduced 9MP processing time from ~50ms down to sub-10ms.

🚀 Future Extensions

- **GPU Acceleration:** Porting core NDWI kernel to CUDA / OpenCL for massive gigapixel satellite rasters.
- **Distributed Cluster Scaling (MPI):** Multi-node distributed processing for historical satellite catalogs.
- **Deep Learning Integration:** Parallel UNet/SegNet neural models for AI flood boundary segmentation.